

MoA as Universal Intermediate Language: Semantic Equivalence, Dimension Lifting, and Portable Deterministic Code Generation

Lenore Mullin — Dissertation 1988 + Papers I-V + Predictive Model Papers + 5-Device Validation

September 30, 2026

Abstract

MoA provides a single universal intermediate representation with semantic equivalence preserved via Theorem 3.1 Storage Theorem. Source programs from Python, PyTorch, JAX, TensorFlow, Fortran, C, Julia, MoA itself compile once to Denotational Normal Form (DNF) — immutable semantic core. DNF is fixed; only Operational Normal Form (ONF) rewrites via gamma transform and dimension lifting produce target-specific optimizations. Dimension lifting maps data shape array $\rho(A)=\langle B,h,n,d,k \rangle$ to machine shape array $\pi=\langle \text{numa.nodes}, \text{cores.per.node}, \text{gpu.warps}, \text{simd.lanes} \rangle$ via partitioning rule $\rho(A) \rightarrow \pi \# (\rho(A) \oslash \pi)$ where $\oslash$ is elementwise division and $\#$ is shape catenation from dissertation. Loops map to portable paradigms ONLY OpenMP, OpenACC, OpenMPI with deterministic shapes and flags -O2 -fno-fast-math -DOMP_DETERMINISTIC fitting $T \approx \alpha F + \beta M \beta$ vs $\alpha 100\text{-}1000 \times \text{DRAM vs FLOP}$, closing prediction gap reached(d) on 5-device ensemble NCSA Delta A100, AMD MI100, PSC Bridges-2 H100, SDSC Expanse V100, TACC Stampede3 Intel Max 1550. Illustrations show AST convergence, shape partitioning, and pragma mapping with gamma start/stop/stride.

1 MoA as Universal Intermediate Language — Many In, Many Out, Semantic Equivalence Preserved

From dissertation Mullin 1988: ψ -selection $\langle \psi \rangle$ with partial indexing $\rho(i \psi xi)=\tau i \text{ drop } \rho xi$, take uparrow, drop downarrow, reverse ϕ , rotate θ , transpose $\oslash$, Lemmas L1-L5 ψ distributes over catenation and unary ops. Ω lifting $f_\Omega(\sigma_l, \sigma_r)$. DNF is unique up to reordering of independent index computations, minimal memory traffic Prop 4.1 $O(nd_k + nd_v)$ vs $O(n^2 + nd_k + nd_v)$.

Figure 1 illustrates MoA as universal IL: many source languages converge to single MoA AST/DNF with semantic equivalence preserved (Theorem 3.1), then diverge to many target backends. DNF is fixed — immutable semantic core. ONF rewrites only via normalized rewrites produce target optimizations without re-derivation. CUDA crossed out — portable only per requirement.

Figure 1: MoA as Universal Intermediate Language: Many Languages In, Many Languages Out. Center MoA AST/DNF with ψ , Ω , γ , compose, semantic equivalence preserved Theorem 3.1. Left: Python, PyTorch, JAX, TensorFlow, Fortran, C, Julia, MoA itself. Right: C+OpenMP, Fortran+OpenACC, OpenMPI, LLVM IR, CUDA crossed out portable only. DNF fixed, ONF rewrites only.

Key: PyTorch Listing 1 `stable_softmax` and attention, JAX `jax.numpy`, Fortran loops all parse to same take/drop/ ψ AST via Def 1.0,1.1,0.0,3.1. Verification Tables 1,2 Paper 1 exact match to $1e-4$, backward Paper 2 —err— $1e-16$, decode Paper 3 —err— 0 exact IEEE-754.

2 Dimension Lifting Using Shapes of Data In and Shapes of Data Out

Dimension lifting Section 3 Paper 1: hardware component modeled as array of known shape P_i . ONF iteration space N mapped to $P_i \# (N \text{ o/ } P_i)$ where o/ elementwise integer division, $\#$ shape catenation (concat) operator from dissertation. Example: hardware p parallel processors arranged as array shape P_i , ONF loop nest iteration space N , dimension lifting maps $N \rightarrow P_i \# (N \text{ o/ } P_i)$, resulting structure expresses inter-processor parallelism outer loop over P_i and intra-processor inner loop over $N \text{ o/ } P_i$ in single algebraic expression.

For higher-rank tensors: B batch, h heads, n sequence, $d.k$ head dim, shape $\{B, h, n, d.k\}$. Transpose over innermost two dims is $(\text{transpose } \Omega_{\{2\}})K$ Eq11, score is $Q((+x) \Omega_{\{2,2\}})((\text{transpose } \Omega_{\{2\}})(K))$ Eq12 distributing over leading $\{B, h\}$ dims. Each 2-D partition assigned to separate processing element GPU warp, SIMD lane, compute node yielding provably correct parallel implementation directly from algebraic structure of Ω without additional scheduling.

Partitioning each dimension of shape array:

$\rho(A) = \{B=8, h=12, n=64, d.k=32\}$ partitioned as $B \rightarrow B_0 \# B_1$ with $8=4 \# 2$, $h \rightarrow h_0 \# h_1$ $12=3 \# 4$, $n \rightarrow n_0 \# n_1$ $64=8 \# 8$, $d.k \rightarrow d.k_0 \# d.k_1$ $32=4 \# 8$ using shape-catenation operator $\#$.

Machine shape $\pi = \{\text{numa_nodes}=2, \text{cores_per_node}=3, \text{gpu_warps}=4, \text{simd_lanes}=8\} = \{2, 3, 4, 8\}$. Elementwise division $\rho(A) \text{ o/ } \pi = \{8/2, 12/3, 64/4, 32/8\} = \{4, 4, 8, 4\}$. Lifted shape: $\rho(A) \rightarrow \pi \# (\rho(A) \text{ o/ } \pi) = \{2, 3, 4, 8\} \# \{4, 4, 8, 4\}$.

Gamma maps multi-index to linear offset: $\gamma(i_B, i_h, i_n, i_{dk}) = ((i^h_0 \pi_1 \pi_2 \pi_3) + (i^h_1 \pi_2 \pi_3) + (i^n_0 \pi_3) - (i^{dk}_0)) * (4 * 4 * 8 * 4) + ((i^h_{14} * 8 * 4) + (i^h_{18} * 4) + (i^h_{14}) + (i^{dk}_{11})).$ Machine - $\text{shape_index_in_}\pi = \langle 2, 3, 4, 8 \rangle$ and $\text{residual_index_in_}\rho(A) \text{ o/ } \pi = \langle 4, 4, 8, 4 \rangle$.

Figure 2: Dimension Lifting using Shape Arrays: Mapping Data Shape $\rho(A) = \{B, h, n, d.k\}$ to Machine Shape $\pi = \{\text{numa_nodes}, \text{cores_per_node}, \text{gpu_warps}, \text{simd_lanes}\}$ via Partitioning and Elementwise Division. Data shape array partitioned along each dimension using shape-catenation operator $\#$. Machine shape array target parallelism. Gamma function maps multi-index to linear offset for execution. Lifted shape $\rho(A) \rightarrow \pi \# (\rho(A) \text{ o/ } \pi)$.

This explains 535x NUMA penalty vs $\{3\}x$ oversubscription Paper V: same DNF, different gamma costs when partitioning mis-aligned to numa_nodes . Also explains C vs Fortran 3.17x time matches 3.35x stall: γ_{row} vs γ_{col} variant.

3 Loops Map to Paradigms — OpenMP Pragmas and Deterministic Shapes

ONF loop nest obtained from DNF by applying gamma primitive Eq3 to all index expressions translating multi-index Cartesian arithmetic into memory offset = base + stride₀*i₀+stride₁*i₁+... Eq7 captures storage convention row-major vs column-major contiguous vs strided enables standard loop-nest analysis.

ONF loops map to portable paradigms ONLY OpenMP, OpenACC, OpenMPI with deterministic shapes to create code. Each pragma maps 1-1 to ONF start/stop/stride, so measured T fits predicted $T = \alpha F + \beta M$ with β/α 100-1000x DRAM vs FLOP closing prediction gap reached(d) on 5-device ensemble.

Figure 3: Loops Map to Paradigms: OpenMP, OpenACC, OpenMPI with Deterministic Shapes. ONF loop nest for i in n_i , k in n_k , j in jdk_i with $\gamma(v,s)$ offset. Mapped to C OpenMP `#pragma omp parallel for schedule(static) proc_bind(close) deterministic shape n_i`, Fortran OpenACC `#pragma acc parallel loop gang worker vector_length(64) independent shape jgang,worker,vector_i`, OpenMPI `MPI_Scatterv/Gatherv shape n0 # n1_i partitioned`. Each loop start/stop/stride from ONF γ , deterministic flags `-O2 -fno-fast-math -DOMP_DETERMINISTIC, -acc -Minfo=accel -ta=tesla:cc80`. All close prediction gap.

Emission examples — deterministic flags only fitting predictive model:

C OpenMP:

```
// MoA DNF->DNF gamma Eq7, C OpenMP deterministic: -O2 -fno-fast-math -fopenmp -
// DOMP_DETERMINISTIC
// rho(Q)=rho(K)=rho(V)=<3,4> Eq9, O(nd_k+nd_v) Prop4.1
#pragma omp parallel for schedule(static) proc_bind(close) // deterministic, fits
// predictive model
for (int i_pp=0; i_pp<3; i_pp++) {
    double q_i[4];
    for (int j=0; j<4; j++) q_i[j]=Q[i_pp*4+j]; // <i'',j>psi Q gamma
    double scores[3];
    for (int k=0; k<3; k++) {
        double s=0.0;
        for (int j=0; j<4; j++) s+=q_i[j]*K[k*4+j]; // Eq17 no K^T buffer
        scores[k]=s/sqrt(4.0);
    }
    // ... softmax Eq16,19,20, Output Eq21
}
```

Fortran OpenACC:

```
! Fortran OpenACC deterministic: -acc -Minfo=accel -ta=tesla:cc80 -fopenacc
// vector_length(64)
!$acc parallel loop gang worker vector_length(64) independent
do i_pp=1,3
    q_i=Q(i_pp,:)
    do k=1,3
        s=0.0d0; do j=1,4; s=s+q_i(j)*K(k,j); end do
        scores(k)=s/sqrt(4.0d0)
    end do
    max_row=maxval(scores); num=exp(scores-max_row); den=sum(num); Y=num/den
    do jv=1,4; Output(i_pp,jv)=sum(Y(:)*V(:,jv)); end do
end do
!$acc end parallel
```

OpenMPI weighted comm-free partition hetero_main: $\rho_{\text{machine}}(d)=n_0 \# n_1_i$:

```
// MPI Scatterv/Gatherv, X_d=0% construction, reached(d) 99.88% busy Level Zero
// Sysman
MPI_Scatterv(Q_global,sendcounts,displs,MPI_DOUBLE,Q_local,n_local*dk,MPI_DOUBLE,0,
MPI_COMM_WORLD);
#pragma omp parallel for schedule(static)
for(int i=0;i<n_local;i++){ /* same DNF local O(nd_k+nd_v) */ }
MPI_Gatherv(Output_local,n_local*dv,MPI_DOUBLE,Output_global,recvcnts,displs,
MPI_DOUBLE,0,MPI_COMM_WORLD);
```

All results: 2.00x atomics -i2.5x speedup via ONF rewrite only Paper V, 535x NUMA vs j3x over-subscription same DNF different γ , C vs Fortran 3.17x time matches 3.35x stall as γ_{row} vs γ_{col} , predictive model closure reached(d) M1exp,Pd,Xd=0%,Cd,Rd 0.004-0.028 measured directly bracket-and-locate sweep Level Zero Sysman 99.88% busy Table X Sec XIII hetero_main+closing-prediction-g

4 Conclusion

MoA as universal IL: many languages in, MoA AST/DNF semantic equivalence preserved Theorem 3.1, many languages out via dimension lifting mapping data shape arrays to machine shape arrays partitioning each dimension using # and o/ and gamma offset. Loops map to portable paradigms OpenMP, OpenACC, OpenMPI with deterministic shapes and flags closing prediction gap. Foundation for exascale.